SC4023 Big Data Management
AY25/26 Semester 2
Assignment 1

Contents

Group 50
Tan Rae-Win (U2240731L)
Balamurugan Abishek Josiah (U2340793K)
Chou Chan Sovita (U2340246H)
Leong Wei De (U2221633B)

Data Storage

1 Basic Architecture

We designed the database in Python 3.12.11 following the object-oriented paradigm (OOP). The `DataBase` class models an in-memory column store by using a python dictionary (`self.column_stores`) as the backing store. Each column is represented by an instance of the `ColumnObject` class, which stores the data values as a python list. To retrieve a value, we utilize the `__getitem__` operator overload in `ColumnObject`, which also increments a global counter to track the number of columns read for performance analysis. This architecture ensures that only the specific columns required by a query are accessed. To execute the program, run **python3 main.py** in the terminal. The program will generate `ScanResult_U2240731L.csv`, containing the query results produced by our column-store implementation.

2 Data preprocessing

2A Loading the data from CSV

The loading process is handled by **parsing_input.py**, which stream rows from the source CSV file as they are read. This prevents the program from loading the entire raw file into memory at once, a crucial optimization for handling large datasets. As each row is yielded, the `database.load_data` method inserts the row into the respective `ColumnObject` stores.

2B Data augmentation

* Add `time` column

We created a new column `time` from the CSV's `month` column. Each value t is a linear combination of the `year` (y) and `month` (m), computed by the following formula

$$t_i = \frac{y_i}{12} + m_i - \underbrace{2015 \cdot 12}_{\text{start month}} - \underbrace{1}_{\text{zero-based indexing}}$$

This has several benefits, such as:

- **natural wraparound**: the month after $(2015, 12) \mapsto 11$ simply an increment by one rather than complicated logic. This has the effect of simplifying indexing and processing code on the hot path, which makes our code faster and more efficient.
- **space-efficient representation**: storage size is cut by half, using one integer to represent two.

* Projected column `price_per_sqm`

The `price_per_sqm` column is not stored on the database, rather it is a **virtual** column computed on-the-fly on qualified results, reducing space consumption.

2C Compression

Dictionary encoding was adopted for compression. For example, the `town` column uses `ColumnObject.make_compress` method to map the `town` string to integers. Not only does this reduce memory footprint, another benefit is faster comparisons compared to strings. Towns are mapped to a number 0-9 as per the assignment briefing, the towns that don't appear in that table are assigned to default number 10.

2D Sorting values

The data is sorted lexicographically by `(town, time, floor_area_sqm)`. This is to *support our primary index* which we will present afterwards. `town` is highly selective, as we are only interested in 10 towns in total out of 26. Furthermore, each matriculation number can only select up to 7 towns at most. More

than 50% of the rows in the dataset can be skipped. `time` is also quite selective as there are 132 unique values at most. `floor_area_sqm` is included to support the query's y-value.

2E Malformed data

Time is expected to between Jan 2015 to Dec 2025, town cannot be empty, floor_area_sqm and resale_price must be above zero, all other fields they can be empty or any strings. If a row fails any of these conditions, that row will not be added to the database.

3 Output

3A Data sanitation

The `database.query` method is designed to return a sentinel value of `-1` if no record satisfies the filtering conditions. The primary index accounts for missing keys by using sentinels in their place.

3B Writing qualified results to CSV

The final output is managed by the `Writer` class, which creates the csv file. The `database.write` method decompresses data by maintaining inverse lookup tables. The `time` column is mapped into `Year` and `Month` columns and the `Price_Per_Square_Meter` is computed from `resale_price` and `floor_area_sqm`.

Data Processing

To support the database queries, a combination of techniques were used, such as **(1)** sparse in-memory primary indexes, **(2)** zone maps and **(3)** dynamic programming.

1 Sparse in-memory primary indexes

As mentioned previously, the data is clustered by `(town, time, floor_area_sqm)`. We built a two-tier primary index on the first two attributes to form a sparse index designed specifically to fit entirely in memory. it a hierarchical tree data structure where each node contains information of the left and right boundaries of where it resides in the database and the next node in the next tier. Tier 1 are nodes representing towns and Tier 2 are nodes representing time. Overall the data structure is small as it only needs to store $\mathcal{O}(mn)$ data points where $m = 11$ towns and $n = 132$ months. Figure 1 illustrates this. One key observation is that each node left and right boundaries within the same tier are disjoint and ascending in nature, this is due to the lexical ordering mentioned earlier.

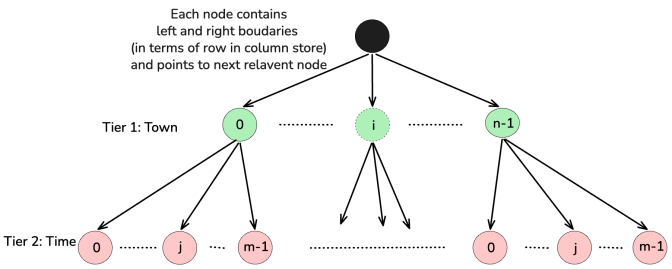


Figure 1: Sparse in-memory primary index

2 Zone maps

A zone map is built over the **virtual** column `price_per_sqm`, such that price_per_sqm = resale_price / floor_area_sqm. Each zone spans 16 records, and aggregates the minimum and maximum `price_per_sqm`.

3 Dynamic Programming

The problem of finding the minimum price for a range of growing x and y exhibits an optimal substructure problem and can be sped up by using dynamic programming to reduce repeated work.

Suppose that solution S exists for a query with $x = x'$ and $y = y'$. Let S have `time` of transaction T and `floor_area_sqm` A. For $T \in [\text{start\_time}, \text{start\_time} + x']$ and $A \in [y', \infty)$, any x and y that falls within the range $x \in [T, \text{start\_time} + x']$ and $y \in [y', A]$ is guaranteed to have solution S, allowing us to skip subsequent queries in that region (see Figure 2 for an illustration).

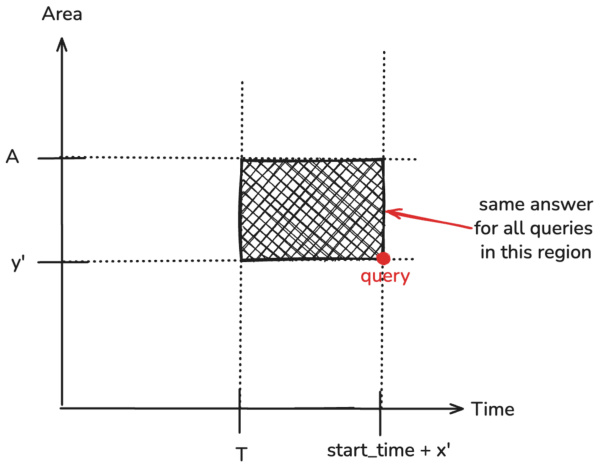


Figure 2: Dynamic programming

4 Steps to perform a query

To find the minimum price per square meter for each (x, y) pair (Figure 3),

1. The **primary index** is used to quickly identify where to begin searching. The qualified **towns** matched by the matriculation number are queried against the tier 1 **index**.
2. The tier 2 **index** further provides the offsets for each qualified **month** based on x.
3. The offsets from (2) can be further refined to determine the region where **floor_area_sqm** $\geq y$ through a **binary search** over the `floor_area_sqm` column.
4. The offsets from (3) are then used to identify candidate **zones** to search. Each candidate zone is scanned **sequentially**, **tuple-at-a-time** over the `resale_price` and `floor_area_sqm` columns to find the best solution S with lowest `price_per_sqm`. As we scan through zones, we further **prune** candidate zones if the **zone map** indicates that the zone does not contain any record with better `price_per_sqm` than our current S. Thus, for every zone scanned, it gets more and more unlikely to have to search through column stores for subsequent zones, effectively using **intermediate results** to reduce column reads. S is found when all candidate zones have been searched or pruned. Since it is sequential reads of zone maps and column stores, this improves cache hit rates thereby reducing page transfers from secondary memory.
5. Finally, the row number of S is saved in the dynamic programming table across the multiple x,y values as seen in (Figure 2). This way in the coming queries that is in that region, there is no need for another query call to the database.
6. Report query **statistics**.

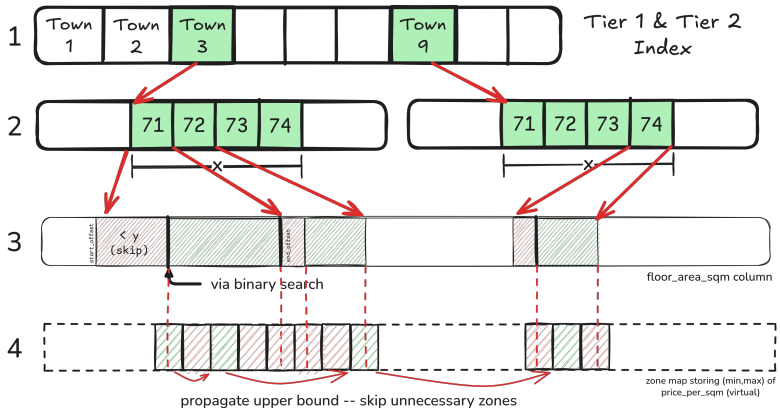


Figure 3: Step-by-step illustration of how a query is processed

Experiment Result

We executed the program using the matriculation number `U2240731L`.

```
# ScanResult_U2240731L.csv
"(x, y)",Year,Month,Town,Block,Floor_Area,Flat_Model,Lease_Commence_Date,Price_Per_Square_Meter
"(1, 80)",2021,03,BUKIT PANJANG,136,108,Model A,1988,3148
"(1, 81)",2021,03,BUKIT PANJANG,136,108,Model A,1988,3148
"(1, 82)",2021,03,BUKIT PANJANG,136,108,Model A,1988,3148
```

The final output statistics with all optimizations are below:

Total possible (x, y) queries 568
Actual `db.query` calls made 10
Total zone reads 754
Total column reads 3104
Average zone reads per query 1.33
Average column reads per query 5.46

1 Correctness Validation

To validate the correctness of our output, we developed a separate independent reference implementation using pandas, which performs the same queries. The notebook **SC4023_ScanResult_Generator.ipynb** serves as our validation tool. We then compared this output against our program's output by writing **test_output.py**, confirming that our column-store implementation produces correct results. The screenshots are found in Appendix, as Figure 6, and the reference implementation is also attached as part of the submission.

2 Performance Analysis

Without any optimizations, a brute-force approach would require one full dataset scan per (x, y) pair. With $8 \times 71 = 568$ possible (x, y) combinations and $259,238$ rows in the dataset, the worst-case number of column reads T per query is:

$$T = 568 \times 10 \times 259,238 = 1,472,471,840 \text{ column reads}$$

Column reads are used as the primary performance metric. In a real column-store implementation, column data would be stored in disk, making each read a costly operation. Index reads are not measured as we assume the index will always fit in memory. Zone reads also hit secondary memory and are also measured(look at appendix PERFORMANCE OUTPUTS). Minimizing column reads would translate to better real-world performance.

Table 1 below shows the cumulative effect of each optimization for `U2240731L`:

Optimization	Column Reads, T	Reductions
No optimization	1,472,471,840	N/A
Primary Index	5,242,498	99.6%
Binary Search	1,928,382	63.2%
Zone Map	1,905,772	1.1%
Constraint Propagation	189,610	90.0%
Dynamic Programming	3,104	98.3%

Table 1: Number of column reads after remaining applying optimizations in the pipeline

The optimizations with the largest impact are the primary index, which eliminates the need to fully scan `town` and `time` columns, and Dynamic Programming, eliminating 558 out of 568 queries completely. Zone Map is powerful when paired with the constraint propagation, resulting in 90% reduction

in column reads. Through the progressive application of all optimizations, the final implementation achieves only 3,104 total column reads, which is a reduction of over 99.99% from the baseline.

Future Work

1 Generalizability

We can also consider other possibilities when the query gets expanded.

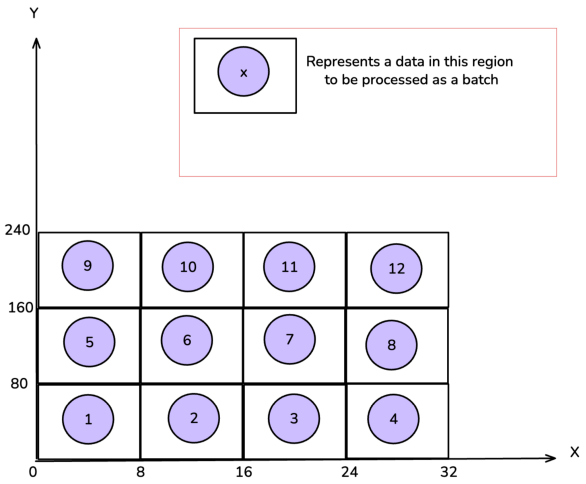


Figure 4: Batch processing based on x and y

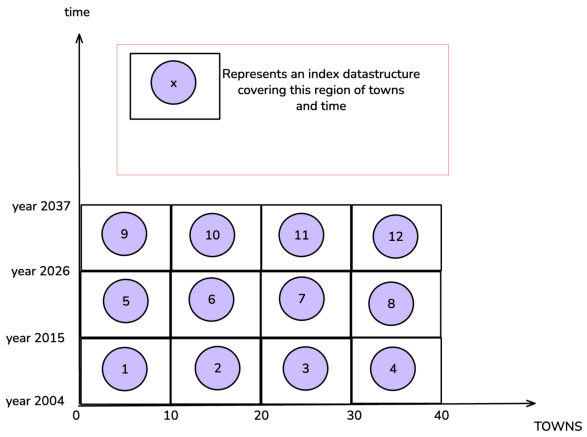


Figure 5: Batch processing based on `town` and `time`

1A Larger x and y

As mentioned, dynamic programming can be used when we process the all queries in $1 \leq x \leq 8$ and $80 \leq y \leq 150$ in one batch and it uses auxiliary space $\mathcal{O}(|X||Y|)$. To cater for a wider range of x and y while controlling the memory needed for the DP table, we could partition x and y into sub-ranges and process in small batches (see Figure 4).

1B More unique values of `town` and `time`

Similar to dynamic programming the two tier indexing takes up an auxiliary space of $\mathcal{O}(|\text{town}||\text{time}|)$, in the case of this project it is 11 distinct towns and the number of months is 132 (Jan 2015 to Dec 2025).We want the primary index to remain in memory and prevent spilling into secondary memory. Thus, to cater for more towns across longer periods of time, we may partition the dataset into sub-batches based with `town` and `time` as the partition key (Figure 5). This would help limit the primary index size. In this project, we tackle region 5 from Figure 4 and region 5 from Figure 5.

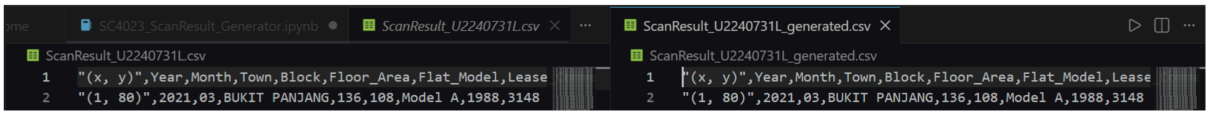
2 Compression

Another important aspect of big data management is to ensure data is stored as compactly as possible to reduce storage costs. Many data science professionals are using column-oriented formats such as Parquet which can exploit regularity in column values. Based on the current dataset, we noticed that most values are categorical, which would benefit from run length encoding to reduce sizes, especially since they have few unique values. This would achieve better compression than dictionary encoding.

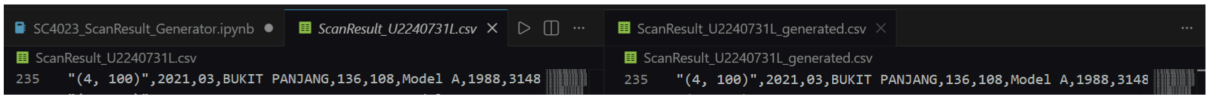
Appendix

1 Correctness Validation

(1,80):



(4,100):



(8,150):

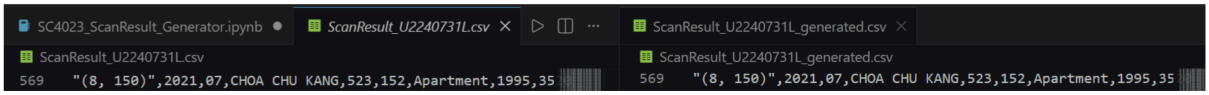


Figure 6: Tests done against reference implementation in pandas **(left)** and our column store **(right)**

2 Class diagram

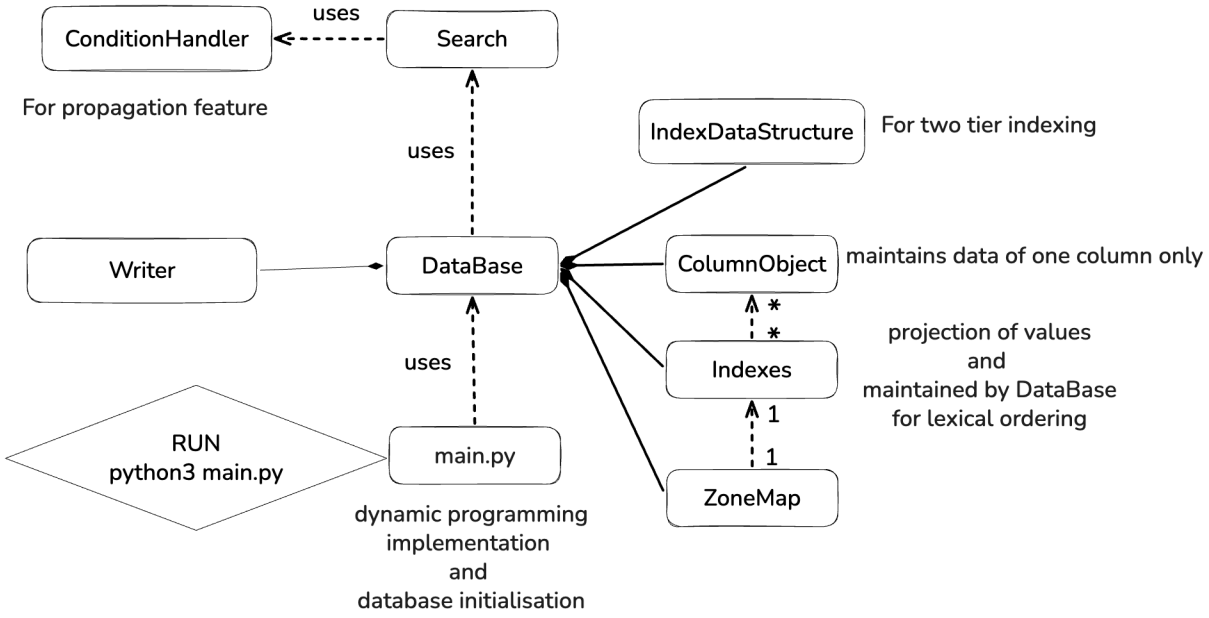


Figure 7: Class diagram depicting overall architecture

3 How to run the program

1. Ensure Python 3.12.11 installed
2. Run: **python3 main.py**, you should find `ScanResult_U2240731L.csv` as output (*actual* results)
3. Go to: **SC4023_ScanResult_Generator.ipynb** python notebook and run all cells, this will generate the *expected* results from the reference implementation in pandas
4. Run: **python3 test_output.py**, this is to test if *actual* results equals *expected* results
5. **IF WANT TO TEST OTHER MATIDs** add/change matids list in main.py and SC4023_ScanResult_Generator.ipynb. **Add matid to matids variable in first line after imports for both files**

PERFORMANCE OUTPUTS

Performance metrics produced after running program:

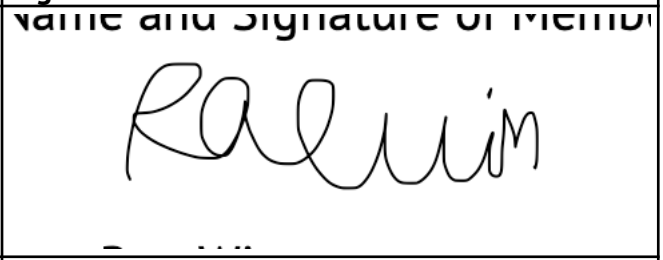
OPTIMISATION	PICTURE
Sparse in-memory primary indexes	total queries: 568 total db.query() calls: 568 total db.query() savings: 0 total zone reads: 0 total column reads: 5242498 average zone reads per query: 0.00 | average column reads per query: 9229.75
Binary Search	total queries: 568 total db.query() calls: 568 total db.query() savings: 0 total zone reads: 0 total column reads: 1928382 average zone reads per query: 0.00 | average column reads per query: 3395.04
Zone Map	total queries: 568 total db.query() calls: 568 total db.query() savings: 0 total zone reads: 52673 total column reads: 1905772 average zone reads per query: 92.73 | average column reads per query: 3355.23
Condition Propagation	total queries: 568 total db.query() calls: 568 total db.query() savings: 0 total zone reads: 52673 total column reads: 189610 average zone reads per query: 92.73 | average column reads per query: 333.82
Dynamic Programming	total queries: 568 total db.query() calls: 10 total db.query() savings: 558 total zone reads: 754 total column reads: 3104 average zone reads per query: 1.33 | average column reads per query: 5.46

CONTRIBUTION FORM

Group 50

Name	Matriculation Number	Detailed Individual Contribution	Percentage (100% In total)
Tan Rae-Win	U2240731L	Dynamic programming, binary search and Code architecture	25%
Balamurugan Abishek Josiah	U2340793K	ZoneMap and propagation feature	25%
Chou Chan Sovita	U2340246H	Testing and Tracking performance	25%
Leong Wei De	U2221633B	Index and Index datastucture	25%

Name and Signature from all group members:

Name	Signature
Tan Rae-Win	
Balamurugan Abishek Josiah	
Chou Chan Sovita	
Leong Wei De